NTNU
Kunnskap for en bedre verden

Department of Computer Science

DT8122 - Probabilistic Artificial Intelligence

Diffusion Model and Bayesian Flow Network Assignment

Author:
Nina Christine Eckertz

September 2024

Table of Contents

1 Introduction

Probabilistic generative artificial intelligence has been on the rise for the generation of images. There are multiple approaches for image generation, e.g., text-to-image with popular models such as DALL-E [13], or image-to-image, e.g., with a diffusion model (DM) [15] or a Bayesian Flow Network (BFN) [3].

This work presents a practical implementation of a DM and further a BFN with an experiment using the MNIST dataset by Yann LeCun [12]. In the following, this report provides an overview of each model type with a methodology section and a discussion. The methodology section of each chapter offers the architecture used. This project is supposed to provide hands-on experience and includes two different approaches for the model setup. Chapter 1 *Diffusion Model* uses pipelines and model architectures from HuggingFace, a commonly used platform for deep learning models and datasets. Chapter 2 *Bayesian Flow Network* uses a native approach using pure PyTorch. The following subchapters provide an overview of the code structure and the dataset.

1.1 Code Structure

The code provided within this project was created using Python and Jupyter Notebook. The models have been trained with Google Colab, with an individual Jupyter notebook for each model.

The notebook is structured after the following components. First, the *Prerequisites* block contains the installation and an import block, which installs all necessary libraries. Second, the *Data* block downloads the MNIST dataset from HuggingFace. In the *Preprocessing* section, the data is prepared by transforming the images to greyscale, resizing and randomly flipping them. This adapts the images for randomisation later and ensures, that the model can process the images as grayscale to boost performance. The BFN notebook contains additional preprocessing in the form of binarisation, which will be explained later on. Third, the block *Model Components* provides the model and its architecture, as well as other components, such as the optimiser, noise scheduler learning rate scheduler and loss function. The BFN further contains an extra block for the setup of the network. Fourth, the *Training* section initialises the training loop. If one provides a pre-trained model, the training will be skipped and the pre-trained model will be provided. Finally, the *Visualisation/Results* section contains some visualisations to evaluate the model performance. The functions to create the visualisations lie in the separate *Visualisation* class.

1.2 Dataset Source

This work uses the MNIST dataset, published by Yann LeCun. The MNIST dataset contains 70,000 28x28 black-and-white images of handwritten digits extracted from two MNIST databases [12]. The MNIST dataset used in this work is obtained from HuggingFace [11], the aforementioned deep learning platform.

2 Diffusion Model

The concept of diffusion implemented in this report was first presented in [15]. In this work, the authors present an approach for deep learning, in which a given data distribution is first systematically destroyed in an iterative forward fusion process. Then, after the destruction, a deep learning model learns to restore the structure of the data in a reverse diffusion process. This mechanism is inspired by non-equilibrium statistical physics and leads to a highly flexible and tractable generative model of the data. The diffusion process continuously adds noise to an image in a series of timesteps, which is discussed in subsection *Diffusion Process and Timestep t*. As mentioned, the model is then trained to reverse the process by predicting the noise added to the image and continuously removing it. This section presents a practical implementation of this concept using the HuggingFace Denoising Diffusion Probabilistic Models (DDPM) pipeline [7].

2.1 Methodology

Based on the work [15], HuggingFace offers a Denoising Diffusion Probabilistic Models (DDPM) pipeline, which refers to both the discrete denoising scheduler as well as the original paper itself [7]. This section provides the practical implementation of this model by discussing the diffusion process and timestep t and the used deep learning model. The DM is based on the HuggingFace documentation [9].

2.1.1 Diffusion Process and Timestep *t*

As described in [15], the forward diffusion process transforms any complex data distribution into a more simple, traceable distribution such as Gaussian using a repeated application of a Markov diffusion kernel. The reverse trajectory simply reverts this process. Within the addition or removal of noise, the amount of to-be-processed noise β depends on the previous iteration of the diffusion process. This makes the DM a Markov chain.

This work uses the HuggingFace DDPMScheduler for the forward diffusion. A `scheduler` is a component that, in the context of DMs, controls how the noise is added to or removed from an image during the diffusion across a series of timesteps t. In the DDPMScheduler timestep t is discrete by default and can be passed as a parameter. If not passed as a parameter, t is set by default to 1000 steps. If set to a higher or consequently lower value, the scheduler adds the noise in fewer or more steps to the image, making it faster or slower. The DDPMScheduler parameter `num_train_timesteps` sets the amount of timesteps t for the diffusion steps. Figure 2 shows the backward trajectory is generated. Figure 1 shows the final result of the image generation providing 16 generated images from the DDPM.

There are further components relevant to the diffusion process and hence to the scheduler. β represents the level of Gaussian noise added to the image. The DDPMScheduler takes β-start and β-end, which define the low-noise and the high-noise value. β-schedule defines the type of schedule for the noise increase. The noise itself is taken from PyTorch *random*, a continuous noise function. Further, as defined by the property `beta_schedule`, this implementation uses the default linear `beta_schedule`.

t is further incorporated into the training process of the neural network, as seen in the source code of the neural network used in this project [8]. To enable the model to know how far the backward trajectory has already gotten, the timestep t is encoded in time embeddings and fed to the neural network during training. The following section discusses the neural network architecture in more detail.

2.1.2 Model Architecture

As suggested by the assignment description this implementation uses an Unet model for the DM. Specifically, this work implements an Unet2D model, an instance of the HuggingFace Unet2D [10]. A U-net is a convolutional neural network, [14], which uses scaling and hence different layers to detect features in an image instead of the widely used sliding window approach. The reason why a DM can use a UNet model lies in the ability to progressively sample its input and hence can operate on different feature abstraction levels. For instance, it can detect more abstract features if the input is sampled down by the model. This is very handy since the noise implemented by the diffusion process can affect the finer and more abstract features of the image.

The following paragraph discusses the structure and features of the implemented HuggingFace Unet2D, based on its source code [8]. The HuggingFace Unet2D is based on the PyTorch Neural Network module. A Unet2D is highly adaptable to its input. Within the initialisation parameters, the `sample_size` specifies the dimensions of the input image. to ensure the alignment of the network's architecture to the input dimension. `in_channels` and `out_channels` defines the number of input and output channels of input and output image. For an RGB image, this is 3, but since the MNIST is a grascale dataset, this implementation uses 1 input and output channel.

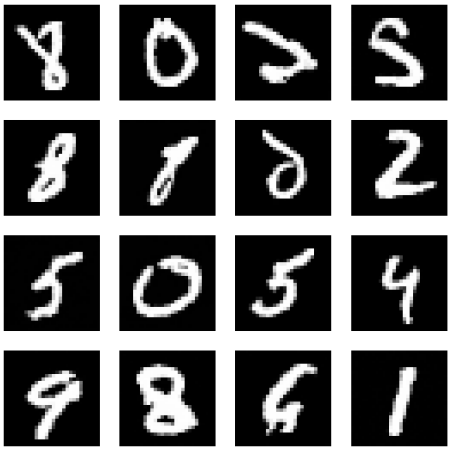

Figure 1: Visualisation of the final result of the DM with 16 generated images. This visualisation was generated using a random sample of MNIST.

Figure 2: Diffusion denoising process, starting with prior.

This type of model consists of two parts: an encoder and a decoder, which mirror one another. In the encoder part, the image is downsampled, whereas in the decoder the image is upsampled. Depending on the problem and data to generate, the model architecture can be complex or fairly simple. In the case of this implementation, the U-net consists of two sampling blocks and one spatial attention block, providing 3 blocks in total to enable a downsampling following 28 x 14 x 7 and the according upsampling. The attention block either within upsampling or downsampling, was incorporated to enable the model to learn the spatial features of the image and to focus on specific areas. This simple architecture was chosen to match the dimensions of the input images and to improve the speed of the training. As mentioned in the previous chapter, not only is the image input but also the time embedding. The time embedding, hence the knowledge of where the model is in the diffusion process helps the model to predict the correct amount of noise.

As for other deep learning problems, there are some more components of relevance. This implementation, following the suggested components by HuggingFace [9] uses an Adam optimiser with weight decay (AdamW) to prevent over-fitting and a learning rate cosine scheduler to dynamically adjust the learning rate during training. Further, the activation function used by this neural network is a Sigmoid Linear Unit (SiLU) function. The loss function mean-squared-error (MSE) loss.

The training function is built upon the HuggingFace accelerator. This is a library that enables training on multiple hardware conditions, for instance, single or multiple GPUs or a CPU. The training configuration uses 41 epochs and a batch size of 64. Figure 3 shows the behaviour of the model throughout training. Here, the advancement of the model is visible, the generated images get better and better. For the last epochs, there is not so much progress in the quality of the generated images visible anymore. This is also noticeable in within the model loss. Figure 4 visualises the development of loss throughout the training. Here, it is noticeable the loss goes down, especially in the first 20 epochs. Afterwards, there is not such a huge development anymore.

2.2 Discussion

2.2.1 Challenges

One challenge has been to modify the model in a way, that it can run under multiple hardware conditions. It is to be mentioned that the model architecture this work provided runs significantly

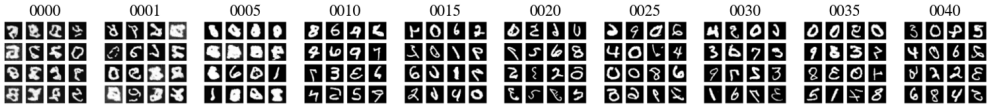

Figure 3: Visualisation of the training of the DM. For the first, last, and every 5th epoch, the model was evaluated on a random batch to see, if the quality of the generated images improves.

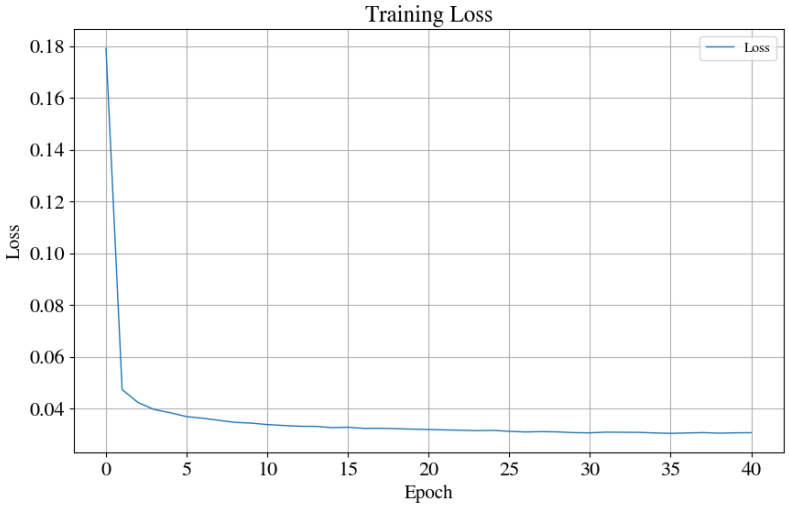

Figure 4: Loss visualisation of the DM. The loss is generally calculated for each step of an epoch, the value per epoch was calculated by averaging those values.

faster with a GPU. This made the training a very time-consuming process, as this needed to work for both models.

Another challenge has been to understand how to work with the HuggingFace diffusion pipeline. With a pipeline or better said frameworks like this, many functionalities and customisation are taken away from the user/implementer. The advantage is that it is easier to achieve a working implementation of an approach and a concept using this framework. On the opposite side, it is also more difficult to understand how the idea, here diffusion, is translated or modelled into code. While I found the set-up of data, model and training very straightforward, the diffusion process and the UNet were harder to understand. Especially when looking into the source code as some aspects were not documented.

3 Task 2: Bayesian Flow Network (BFN)

The BFN is another kind of generative model and was first introduced in [3]. This model learns to generate data by optimising the parameters of a given data distribution, instead of working with the data samples directly like a DM. Nonetheless, it is linked to DM. Through the operation in the distribution instead of the data space, the BFN can handle multiple types of data: continuous data, discretised continuous and discrete data. Hence, it can not only generate images but also, e.g., text.

The model parameter optimisation is inspired by an exchange between a sender and a receiver, who iteratively exchange information about a data distribution. On one hand, the sender sends information about the data to the receiver. Here, the sender adds noise to the data distribution and sends a sample to the receiver, creating a sender distribution. On the other hand, the receiver wishes to receive the information with low transmission costs, hence the receiver tries to forecast the next message by the receiver to lower the transmission costs. The receiver also sends his forecast to a neural network, which then outputs a new output distribution with updated parameters. Further, the receiver creates a receiver distribution by mixing the sender's noise with the output

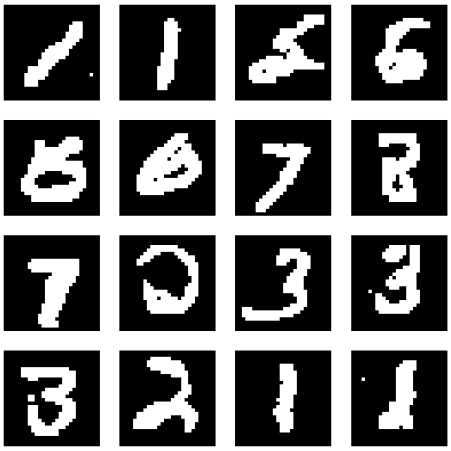

Figure 5: Final results of the image generation using a BFN. This was generated using a random sample of the data.

Figure 6: Denoising of the BFN across 10 steps.

distribution. The transmission cost is measured between the sender and receiver distribution with KL divergence. By repetition of the aforementioned process, the receiver can lower the transmission costs, hence lowering the loss, eventually.

3.1 Methodology

In this project, the BFN was implemented following an example by [2]. The implementation of the BFN in this work follows a similar implementation structure as the previous DM wit the same MNIST dataset. However, this implementation uses a direct implementation based on PyTorch using an example from GitHub [2].

3.1.1 Data Preprocessing

The general preprocessing of the dataset is analogue to the preprocessing in the DM. Since the flexibility of a BFN offers to work not only with continuous but also discretised continuous and discrete data, this implementation tries discretised continuous data. Binarisation of characters is a common preprocessing technique in OCR, for instance, to enhance the readability of a text in document processing. [4] The data is binarised in function `collate_dynamic-binarize`, in which each batch of the dataset is binarized. The grayscale pixel intensities of every image in a batch are treated as a Bernoulli probability and sampled through a particular binarisation threshold which is held fixed during training [2]. The dynamic binarisation sets a novel threshold for every batch. This is achieved through a comparison of the image, represented as a normalised tensor, and random threshold value from a uniform distribution between 0 and 1 using `torch.rand()`. This puts the pixels within the image 'on' or 'off', resulting in a discrete representation of the images. This is also the reason, why the generated images in the results of the BFN 5 appear not as 'soft' in comparison to 1, despite using the same graphic styling. The binarisation further leads K to 2.

Figure 7: Visualisation of the training of the BFN. For the first, last, and every 5th epoch, the model was evaluated on a random batch to see, if the quality of the generated images improved.

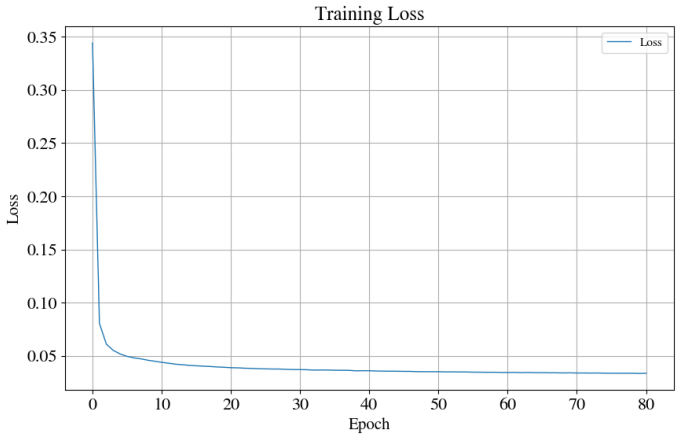

Figure 8: Loss visualisation of the BFN. The loss is generally calculated for each step of an epoch, the value per epoch was calculated by averaging those values.

3.1.2 Model Architecture

Similar to the implementation of the DM, this implementation also uses a simplified U-Net [2]. According to [2] and as seen in the code, the model has two residual blocks going from 64 to 256 feature maps in the inner layer for a total of 2,8M parameters. Hence, the model follows a structure with a three-layered encoder, and a three-layered decoder to match the input and output dimensions of the dataset. In contrast to the previously discussed implementation DM, this implementation sets up the Unet model from scratch using PyTorch `nn_module`.

For the BFN implementation, two functions are essential: `process` and `sample`. `process` is the function to train the model and consists of five steps, which follows the process as described in [3] [2]. First, the network samples the timestep t from a uniform distribution in the unit range. Second, it then computes β, the accuracy schedule, based on t. Third, the network samples y from a Gaussian distribution centred on the image and with standard deviation inverse proportional to t. Fourth, the network computes θ, the input distribution's parameters, by taking the softmax of y (rescaling it in the unit interval). Last, the network sends θ and t to the UNet to obtain the output distribution's parameters. Once output parameters are obtained, one can compute the loss [2]. Within the `sample` function, the network first initialises θ as $\frac{1}{K}$, representing the parameters of a uniform prior. Second, it defines $t = \frac{\text{step}-1}{n_{\text{step}}}$ and starts iterating through all sampling steps. Third, the network samples k, the generated batch, from the output distribution obtained by passing θ and t to the network. Fourth, it samples y from a Gaussian distribution centered on k, with standard deviation inversely proportional to the current sampling step. Fifth, the network updates θ by applying softmax to $y \cdot \log(\theta)$. Finally, once the loop finishes, it samples and returns k from the output distribution obtained by passing θ and t to the network [2]. Further, the implementation of the BFN uses an AdamW optimizer with the hyper-parameters like in the paper [2]. In this implementation, an exponential moving average (EMA) of the parameters was applied for the sample generation [2]. For the general training configuration, such as `batch_size` and `num_of_epochs` a similar configuration as for the DM was used, hence a `batch_size` of 64, but 81 epochs to adjust for the different amount of steps of each epoch.

Similarly to the DM, the training function for the BFN is built upon the HuggingFace accelerator to enable more efficient training on GPU. The training configuration uses 81 epochs and a batch size of 64. Figure 7 shows the behaviour of the model throughout training. Here, the advancement of the model is visible, the generated images get better and better. For the last epochs, there is not

so much progress in the quality of the generated images visible anymore. This is also noticeable in within the model loss. Figure 8 visualises the development of loss throughout the training. Here, it is noticeable the loss goes down, especially in the first 30 epochs. Afterwards, there is not such a huge development anymore.

3.2 Discussion

3.2.1 Similarities and Differences of DM and BFN. How do the methods compare when learning discrete distributions?

Both DM and BFN are generative probabilistic models with the objective generate images, hence it makes sense to compare them as the objective is the same. As stated in [3], Diffusion and BFN are similar, but they share also some major differences.

A major difference between the models is the approach to generate the image. The DM works with the data directly, by creating a noised version of an image and learning to reconstruct the original image by predicting the noise created in the forward trajectory. Hence, a DM takes data as input and outputs the distribution. As the noise is added and removed in a Markov chain, this means that each noise state depends on the previous state and hence the diffusion is continuous by nature. Even though DMs were also applied to discrete data as seen in [5] and [15], this corruption process, due to the usage of continuous, Gaussian noise, is much for fit for continuous data that does not operate with absolute values such as text processing.

In contrast, the BFN takes a distribution as an input and also outputs a distribution. For each training iteration, the model generates a little bit more information about the original data distribution. The information about the original data distribution comes as a noisy sample drawn from the data. The model learns then to remove the noise, The lower the received noise, the model updates the parameters of the data distribution. Hence, one major difference between diffusion and BFN models is that the DM works directly with the data, and one operates in the distribution space, a representation of the data. This provides the BFN flexibility to handle continuous, discretised continuous and discrete data. Also, this is why the BFN does not require any forward diffusion of the data.

To dive deeper into the topic of DMs and discrete distributions, recent research has made some interesting propositions to solve the issue. [6] provides two extensions to generative flows and DM through Argmax Flows and Multinominal Diffusion to enable the model to learn categorical data, e.g., text. Argmax Flows are composed of an Argmax function and the composition of a continuous distribution, e.g., a normalising flow. Further, Multinomial Diffusion involves the incremental addition of categorical noise during the diffusion process. Hence, the model learns a generative denoising process based on the categorical noise. This further confirms that the use of continuous, Gaussian noise is therefore another reason why the 'regular' DM [15] can not handle discrete data. Also, [1] made an interesting contribution to discrete DMs through Discrete Denoising Diffusion Probabilistic Models (D3PMs). This model generalises the Multinomal Diffusion by [6], by introducing a corruption process beyond uniform transition probabilities. Here, the authors use, for instance, transition matrices that mimic Gaussian kernels in continuous space and matrices based on nearest neighbours in embedding space. The authors conclude, that the choice of transition matrix is an important design decision that improves the results in image and text generation. With the recent publication of [16] there is also a first case study to unify BFN and DM through stochastic differential equations (SDEs). Furthermore, the authors stress that the relationship between BFNs and DM is not been fully explored yet, so it is assumed, that future research could publish other approaches.

Within practical work, one similarity lies in using deep learning model architecture for image generation. As seen in the provided code examples, for both tasks a Unet architecture with similar dimensions to fit in the input media. Hence, the encoder and decoder methods are similar.

3.2.2 Compare the Results with the Diffusion Results. How do the different methods perform?

Figure 5 and 1 provide an example of the results. Here, the DM results appear to be slightly better. Generally, both models were able to generate readable digits. Within the denoising trajectory, it is further noticeable, that the structure of the DM sample has been corrupted more.

In terms of performance, both models were trained with different steps per epoch, alternating the amount of training data. The training of the BFN can be still assumed to be faster, since the BFN only does not perform a forward diffusion and, due to the binarisation, per less resource-intensive image processing.

3.2.3 Can a fair comparison of the results be made because of your implementation?

Generally, since both BFN and DM solve the same image generation problem, a comparison can be useful. However, a comparison would be probably not fair based on this work. This has several reasons. First, the approach to image processing a different. The DM works directly with continuous images of the dataset, whereas the BFN operates with a binarised representation, leading to different paradigms on the same content. Second, using a library like HuggingFace can limit reproducibility and control, in contrast to the custom BFN, as one can not edit or modify the library's source code easily, if even at all. Hence, there is a dependence for libraries on specific versions and changes made by the authors of a given library.

However, some aspects can enable a fair comparison of the model. The code structure for both models provides an overview so that it is clear which calculations or parts of data processing are carried out. Another important aspect to highlight is the similar training configuration of using the same `batch_size` to establish the specific frame for the training. The same applies to the choice to use the same dataset from the same source. Also, the visualisation function was constipated to fit both model types. So, for a more fair comparison in a future version of this work, both models should not only feature the same general training configuration but also the same amount of steps per epoch and the same number of epochs. Most importantly they should work both in the continuous space and feature the same preprocessing process.

4 Conclusion

To conclude, this work presents practical implementations of a diffusion model (DM) [15] and Bayesian flow network (BFN) [3]. While both model types can generate images as shown in this work, their approach is different. On the one hand, the diffusion model works directly with the data by systematically destroying their structure and learning to reconstruct them. On the other hand, the BFN operates in distribution spaces and finetunes the network parameters instead of working directly with the data. The BFN therefore offers more flexibility concerning the type of its input data and can learn from continuous, discretised continuous and discrete data. The diffusion model with its continuous data processing is best suited for continuous data such as images, in contrast to data that operates on absolute states such as text. Although, there is ongoing research to tackle this, for instance through Discrete Denoising Diffusion Probabilistic Models (D3PMs) [1] or the fusion of DM and BFN [16].

Future improvements in this practical implementation can be made in terms of the comparability of both models. For instance through the manual set-up of the DM without the HuggingFace platform or through the implementation of continuous image data as input to the BFN.

Bibliography

[1] Jacob Austin et al. 'Structured Denoising Diffusion Models in Discrete State-Spaces'. In: *Advances in Neural Information Processing Systems*. Vol. 34. Curran Associates, Inc., 2021, pp. 17981–17993. URL: https://proceedings.neurips.cc/paper/2021/hash/958c530554f78bcd8e97125b70e6973d-Abstract.html (visited on 12th Sept. 2024).

[2] Davide Ghilardi. *Image classification with Bayesian Flow Networks in Python*. en. Sept. 2023. URL: https://medium.com/@ddxzzx/image-classification-with-bayesian-flow-networks-in-python-d83342e1374 (visited on 19th Aug. 2024).

[3] Alex Graves et al. *Bayesian Flow Networks*. en. arXiv:2308.07037 [cs]. Feb. 2024. URL: http://arxiv.org/abs/2308.07037 (visited on 5th Aug. 2024).

[4] Maya R. Gupta, Nathaniel P. Jacobson and Eric K. Garcia. 'OCR binarization and image pre-processing for searching historical documents'. en. In: *Pattern Recognition* 40.2 (Feb. 2007), pp. 389–397. ISSN: 00313203. DOI: 10.1016/j.patcog.2006.04.043. URL: https://linkinghub.elsevier.com/retrieve/pii/S0031320306002202 (visited on 13th Sept. 2024).

[5] Jonathan Ho, Ajay Jain and Pieter Abbeel. 'Denoising Diffusion Probabilistic Models'. en. In: (2020).

[6] Emiel Hoogeboom et al. 'Argmax Flows and Multinomial Diffusion: Learning Categorical Distributions'. en. In: (2021).

[7] HuggingFace. *DDPM*. 2024. URL: https://huggingface.co/docs/diffusers/api/pipelines/ddpm (visited on 9th Sept. 2024).

[8] HuggingFace. *src/diffusers/models/unets/unet_2d.py*. 2024. URL: https://github.com/huggingface/diffusers/blob/v0.30.2/src/diffusers/models/unets/unet_2d.py#L40 (visited on 9th Sept. 2024).

[9] HuggingFace. *Train a diffusion model*. 2024. URL: https://huggingface.co/docs/diffusers/tutorials/basic_training (visited on 9th Sept. 2024).

[10] HuggingFace. *UNet2DModel*. 2024. URL: https://huggingface.co/docs/diffusers/api/models/unet2d (visited on 9th Sept. 2024).

[11] Y. Lecun. *MNIST Dataset*. Aug. 2022. URL: https://huggingface.co/datasets/ylecun/mnist (visited on 13th Sept. 2024).

[12] Y. Lecun et al. 'Gradient-based learning applied to document recognition'. In: *Proceedings of the IEEE* 86.11 (Nov. 1998), pp. 2278–2324. ISSN: 00189219. DOI: 10.1109/5.726791. URL: http://ieeexplore.ieee.org/document/726791/ (visited on 5th Aug. 2024).

[13] Aditya Ramesh et al. *Zero-Shot Text-to-Image Generation*. en. arXiv:2102.12092 [cs]. Feb. 2021. URL: http://arxiv.org/abs/2102.12092 (visited on 13th Sept. 2024).

[14] Olaf Ronneberger, Philipp Fischer and Thomas Brox. 'U-Net: Convolutional Networks for Biomedical Image Segmentation'. In: *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015*. Ed. by Nassir Navab et al. Cham: Springer International Publishing, 2015, pp. 234–241. ISBN: 978-3-319-24574-4.

[15] Jascha Sohl-Dickstein et al. *Deep Unsupervised Learning using Nonequilibrium Thermodynamics*. en. arXiv:1503.03585 [cond-mat, q-bio, stat]. Nov. 2015. URL: http://arxiv.org/abs/1503.03585 (visited on 5th Aug. 2024).

[16] Kaiwen Xue et al. *Unifying Bayesian Flow Networks and Diffusion Models through Stochastic Differential Equations*. en. arXiv:2404.15766 [cs]. June 2024. URL: http://arxiv.org/abs/2404.15766 (visited on 13th Sept. 2024).